

Weekly Report: Partial DAG Prediction for Dyserve

Kun Lin

August 26, 2026

1 What I Did This Week

This week, I continued working on dynamic DAG prediction for Dyserve. The main goal is to use historical agent traces to predict how the current workflow may change in the next few steps. If the future structure can be predicted early, Dyserve may be able to prepare the ILP computation before the new DAG is fully available.

The work has two parts:

1. Convert raw SWE-Trace trajectories into approximate DAG data.
2. Use the currently visible partial DAG to predict a limited set of future node counts and topology classes.

The current experiment uses 1,000 OpenHands traces from Open-SWE-Traces.

2 Extracting DAG Data from SWE-Trace

The raw SWE-Trace data contains complete tool calls instead of only simplified labels. For each call, the trace may include the tool name, command, file path, tool result, and whether the operation succeeds or fails.

A simplified example is shown below.

Step	Action	File or resource	Result
1	Inspect	<code>src/app.py</code>	Success
2	Inspect	<code>tests/test_app.py</code>	Success
3	Edit	<code>src/app.py</code>	Success
4	Test	Test suite	Failed
5	Inspect	<code>src/app.py</code>	Success
6	Edit	<code>src/app.py</code>	Success
7	Test	Test suite	Success

Each tool call is converted into one graph node. A node stores its semantic type, the tool used, resources read or written, execution result, and discovery position.

The trace does not provide real workflow edges, so I infer dependencies using a small set of rules:

- Inspecting a file before editing it creates a read-to-write dependency.
- Editing a file before reading it again creates a write-to-read dependency.
- Two edits to the same file create a write-to-write dependency.
- Tests are connected to recent file edits that they may validate.

- A failed operation followed by more inspection or editing creates a possible recovery path.
- Calls that are only next to each other in time receive a low-confidence relationship.

Each inferred edge has a confidence score. High-confidence file dependencies are kept during training, while weak temporal relationships can be removed. The result is an approximate tool and data-dependency DAG. It is useful for this early experiment, but it is not the same as a ground-truth Dyserve DAG exported by a workflow orchestrator.

3 Creating Prediction Samples

Each DAG is replayed in the order that its nodes appear. At every position, the model only receives the nodes and edges that are visible at that time. The future graph is used only to create the prediction target.

The current prediction horizon is three nodes. For example, the model may see a partial DAG containing two inspect nodes and one edit node. The next three nodes may contain one inspect, one edit, and one test, arranged as a chain.

One trace produces many training samples because a prediction sample is created at almost every execution position. The 1,000 traces produce:

- 39,629 training positions;
- 7,174 validation positions;
- 11,858 test positions.

The split is based on workflow instance, so positions from the same task cannot appear in both the training and test sets.

4 Current Prediction Model

The current model uses bounded outputs instead of trying to generate an arbitrary future graph. The prediction is divided into two parts.

4.1 Future Node Counts

For each semantic node type, one classifier predicts how many times it appears in the next three positions. The possible outputs are 0, 1, 2, or 3. The current node types are inspect, edit, shell, test, reason, web, and other.

For example, one prediction may be:

$$\text{inspect} = 1, \text{edit} = 1, \text{test} = 1, \text{all other types} = 0.$$

This keeps the output space fixed when the dataset grows. It also allows each node type to be evaluated separately.

4.2 Future Topology

A separate classifier predicts the rough topology of the next few nodes. The possible classes are:

- independent;
- chain;
- branch;
- join;
- branch followed by join;

- no future change.

The model does not predict exact future node IDs or every edge. It predicts a small amount of structural information that may later be useful for preparing an ILP candidate.

4.3 Input Features

The model currently uses CatBoost. The partial DAG is summarized into simple features such as:

- recent action types;
- latest tool and success/failure state;
- number of visible nodes and edges;
- current graph depth;
- number of branches and joins;
- maximum fan-in and fan-out;
- counts of different semantic node types; and
- programming language.

I compare three input settings:

Sequence: recent action history only;

Graph: partial-DAG statistics only;

Combined: both sequence and graph information.

5 Results on 1,000 Traces

5.1 Topology Prediction

Input	Accuracy	Macro-F1	Top-3 coverage
Sequence	45.59%	31.19%	88.26%
Graph	45.27%	31.08%	88.72%
Combined	47.74%	34.51%	89.43%
Majority baseline	30.31%	—	—

The combined model performs best. Its topology accuracy is about 2.15 percentage points higher than the sequence model and 17.43 points higher than always predicting the most common topology.

The model performs best on branch-and-join and independent structures. However, it does not correctly identify the rare branch-only class. Many branch and join samples are incorrectly predicted as branch-and-join. This is currently the main topology prediction problem.

5.2 Node Presence Prediction

For dynamic prediction, it is useful to know whether a type of node will appear, even when the exact count is not correct.

Input	Presence Micro-F1	Presence Macro-F1	Exact count vector
Sequence	61.11%	29.91%	20.35%
Graph	58.36%	26.33%	18.23%
Combined	63.21%	31.68%	20.68%

The combined model again performs best. The result suggests that graph statistics provide some additional information, but recent execution history is still the main signal.

The combined model’s presence F1 scores for common node types are:

Node type	Precision	Recall	F1
Inspect	83.07%	80.59%	81.81%
Edit	72.54%	40.29%	51.81%
Shell	70.48%	44.60%	54.63%
Test	67.70%	17.32%	27.58%
Reason	0%	0%	0%

Inspect is relatively easy to predict. Edit and shell have reasonable precision but still miss many real events. Test recall is low, and reason is too rare for the current default decision rule to predict.

5.3 Strict Complete Match

The combined model predicts all node counts correctly in 20.68% of test positions. It predicts all counts and topology correctly in 12.88% of positions. The majority baseline reaches 26.96% and 14.62% on these two very strict metrics.

This happens because the majority baseline repeatedly predicts the most common count pattern, while the learned model attempts to detect more edit, shell, and test events. The learned model has better event F1 and topology prediction, but it makes more small count errors. For the current dynamic DAG task, I treat topology Macro-F1 and node presence F1 as the main metrics, while complete match is kept as a strict secondary metric.

6 Results on 10,000 Traces

The larger experiment uses 10,000 workflows. It contains 400,448 training positions, 69,620 validation positions, and 116,677 test positions.

6.1 Topology Results

Input	Accuracy	Macro-F1	Top-3 coverage
Majority baseline	30.62%	—	—
Sequence	46.32%	31.79%	88.91%
Graph	46.98%	31.98%	89.57%
Combined	49.31%	35.51%	90.34%

Combined remains the best model. It improves topology accuracy by 2.99 percentage points and Macro-F1 by 3.72 points over Sequence. Compared with the 1,000-trace experiment, Combined topology accuracy increases from 47.74% to 49.31%, while Top-3 coverage increases from 89.43% to 90.34%.

The branch-only class remains difficult. Its recall increases from zero to only 0.32%. Many branch and join samples are still predicted as branch-and-join.

Input	Presence Micro-F1	Presence Macro-F1	Exact count vector
Sequence	61.53%	30.25%	21.19%
Graph	57.85%	25.96%	18.89%
Combined	63.11%	32.04%	21.41%

6.2 Node Presence and Count Results

Combined again performs best. Shell prediction improves the most, reaching 56.29% F1 and 73.08% AUPRC. Inspect remains the easiest event, with 81.22% F1. Test recall is only 17.63%, and reason is still never selected by the default count decision rule.

6.3 Comparison with 1,000 Traces

Combined metric	1K	10K	Change
Topology Accuracy	47.74%	49.31%	+1.57 pp
Topology Macro-F1	34.51%	35.51%	+1.00 pp
Top-3 Coverage	89.43%	90.34%	+0.91 pp
Presence Micro-F1	63.21%	63.11%	-0.10 pp
Presence Macro-F1	31.68%	32.04%	+0.36 pp
Complete Exact Match	12.88%	13.68%	+0.80 pp

More data mainly improves topology prediction. Node-presence performance stays almost unchanged, suggesting that it is close to the limit of the current features and decision rules.

The strict complete match is 13.68% for Combined, compared with a 15.12% majority baseline. The majority predictor benefits from repeatedly choosing the most common count pattern. Combined detects more non-majority events, improving event F1 and topology prediction while making more small count errors.

7 What the Results Show

The current results show three main points:

1. Sequence information is still the strongest single source of information.
2. Graph features are not strong enough by themselves, but they improve the sequence model when both are combined.
3. The bounded output design is easier to analyze because topology, node presence, and node counts can be evaluated separately.

The improvement from graph features is positive but still small. The 10,000- trace result follows the same ordering as the 1,000-trace result, which provides stronger evidence that the improvement is real. Multiple random seeds are still needed to measure its variance.

8 How This May Help Dyserve

The current model is not accurate enough to generate and directly execute a complete future DAG. At 10K scale, topology accuracy is 49.31%, and all node counts and topology are correct together

in only 13.68% of test positions. Therefore, a predicted ILP solution should not replace the real solution before the actual DAG is known.

However, the model may still help Dyserve with low-risk preparation. The top-three topology coverage is 90.34%, and node-presence Micro-F1 is 63.11%. This means the model often places the real coarse structure among a small number of candidates, even when the exact future graph is wrong.

The current model may be useful for the following tasks:

- load model and verifier profiles before the future nodes appear;
- prepare candidate variables and shared ILP constraints;
- create a small number of topology templates;
- copy the current ILP solution as a possible warm start;
- prepare an incremental or residual ILP problem; and
- start speculative preparation only when prediction confidence is high.

A possible integration is:

Current partial DAG → predictor → top candidate structures → prepare ILP templates and warm starts → real DAG appears → verify and reuse matching work.

If the prediction matches the real DAG, Dyserve can reuse the prepared template or warm start. If it does not match, the candidate is discarded and Dyserve uses the normal compilation path. The real DAG is always checked before a plan is committed.

This is safer than running one predicted ILP result as the final plan. The model currently misses rare branch-only structures, so it should not be used as the only detector of workflow expansion. It is better treated as an optional high-confidence gate or preparation assistant.

The expected benefit is to hide part of the ILP setup and solve time rather than remove the whole compilation cost. The current SWE-Trace experiment cannot yet measure this benefit. A Dyserve integration experiment still needs to record:

- how early the prediction is made;
- how much ILP work is reused;
- how much solver time is hidden;
- how many speculative candidates are discarded;
- prediction and preparation overhead; and
- end-to-end workflow latency.

Therefore, the current result supports trying a limited Dyserve integration, especially ILP template preparation and warm starts. It does not yet show that the model can safely control the final runtime plan.

9 Current Problems and Next Steps

The main remaining problems are class imbalance and noisy inferred edges. The branch class is rare and is never selected by the current topology model. The count models also prefer zero for rare node types, which causes low recall.

The next steps are:

1. add class weights for rare topology classes, especially branch and join;
2. choose a separate presence threshold for each node type using the validation set;
3. separate event presence prediction from positive count prediction;
4. inspect and separate termination or unknown nodes currently grouped as other;
5. repeat the experiment with multiple random seeds;

6. later connect these predictions to whether a real Dyserve ILP solution changes or can be reused.

10 Summary

This week, I converted raw SWE-Trace trajectories into inferred partial DAGs and trained a bounded future-DAG predictor. The model separately predicts future node counts and coarse topology. The 1,000- and 10,000-trace experiments both show that combining sequence and graph features gives the best results. At 10K scale, Combined reaches 49.31% topology accuracy, 35.51% topology Macro-F1, 90.34% Top-3 coverage, and 63.11% Presence Micro-F1. The gain from graph features is consistent but limited, and rare branches and events remain difficult. The next work will focus on class imbalance, threshold selection, multiple random seeds, and Dyserve integration.